

Nama : Deswita Khansa Rafifah

NIM : 254107020151

Kelas : TI-1G

LAPORAN JOBSHEET 5

5.2 Percobaan 1: Menghitung Nilai Faktorial dengan Algoritma Brute Force dan Divide and Conquer

Kode program:

```
package BruteForceDivideConquer.minggu5;

public class Faktorial06 {
    int faktorialBF(int n) {
        int fakto = 1;
        for(int i = 1; i <= n; i++) {
            fakto = fakto * i;
        }
        return fakto;
    }

    int faktorialDC(int n) {
        if (n == 1) {
            return 1;
        } else {
            int fakto = n * faktorialDC(n - 1);
            return fakto;
        }
    }
}

package BruteForceDivideConquer.minggu5;
import java.util.Scanner;

public class MainFaktorial06 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan nilai: ");
        int nilai = input.nextInt();

        Faktorial06 fk = new Faktorial06();
        System.out.println("Nilai faktorial " + nilai + " menggunakan
BF: " + fk.faktorialBF(nilai));
    }
}
```

```

        System.out.println("Nilai faktorial " + nilai + " menggunakan
DC: " + fk.faktorialDC(nilai));
    }
}

```

Hasil running:

```

Masukkan nilai: 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
PS C:\Users\Asus\Documents\Algoritma dan Struktur Data\PraktikumASD>

```

Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!

Jawab:

- **if (n==1)** → base case (kondisi berhenti). Ketika n sudah mencapai 1, rekursi berhenti dan langsung mengembalikan nilai 1, karena $1! = 1$.
- **else** → recursive case. Ketika $n > 1$, fungsi memanggil dirinya sendiri dengan nilai $n-1$, lalu mengalikan hasilnya dengan n. Proses ini terus berulang hingga mencapai base case.

2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!

Jawab: Bisa. Prinsip Brute Force adalah menyelesaikan masalah secara langsung dan sederhana, sehingga jenis perulangan yang digunakan tidak mempengaruhi prinsipnya selama proses tetap berjalan satu per satu. Berikut buktinya menggunakan while dan do-while:

→ Menggunakan while

```

BruteForceDivideConquer > minggu5 > J Faktorial06.java > Faktorial06 > faktorialBF_while(int)
1 package BruteForceDivideConquer.minggu5;
2
3 public class Faktorial06 {
4     int faktorialBF_while(int n) {
5         int fakto = 1;
6         int i = 1;
7         while (i <= n) {
8             fakto = fakto * i;
9             i++;
10        }
11        return fakto;
12    }
13 }

```

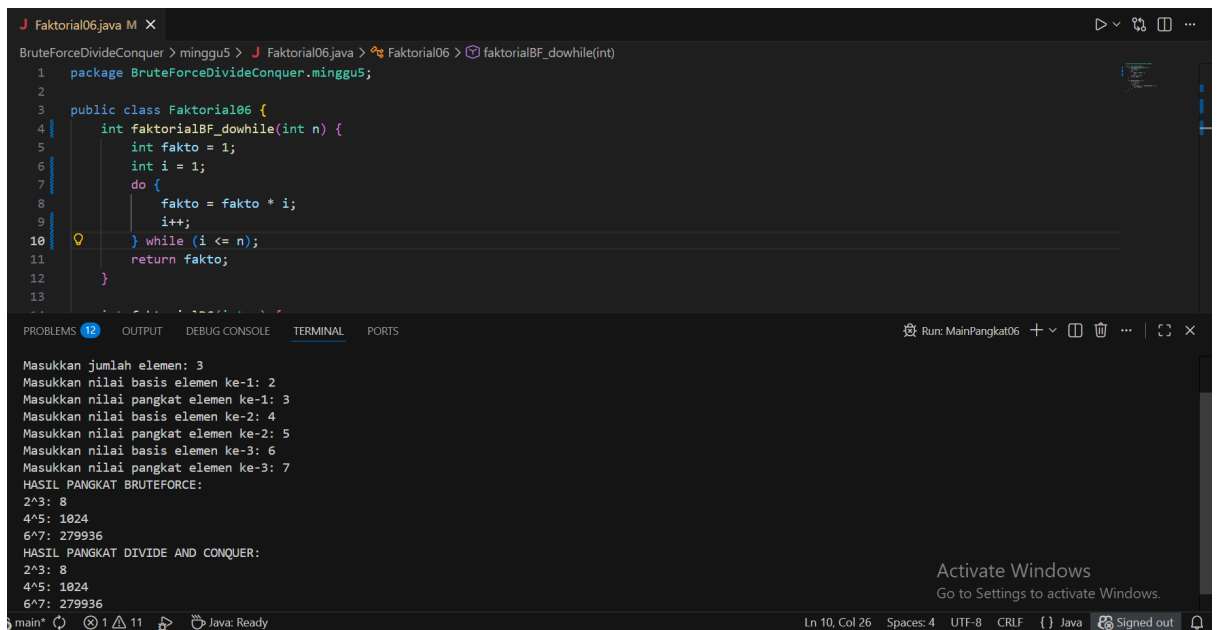
TERMINAL OUTPUT:

```

Masukkan jumlah elemen: 3
Masukkan nilai basis elemen ke-1: 2
Masukkan nilai pangkat elemen ke-1: 3
Masukkan nilai basis elemen ke-2: 4
Masukkan nilai pangkat elemen ke-2: 5
Masukkan nilai basis elemen ke-3: 6
Masukkan nilai pangkat elemen ke-3: 7
HASIL PANGKAT BRUTEFORCE:
2^3: 8
4^5: 1024
6^7: 279936
HASIL PANGKAT DIVIDE AND CONQUER:
2^3: 8
4^5: 1024
6^7: 279936

```

→ Menggunakan do-while



```
BruteForceDivideConquer > minggu5 > J Faktorial06.java > Faktorial06 > faktorialBF_dowhile(int)
1 package BruteForceDivideConquer.minggu5;
2
3 public class Faktorial06 {
4     int faktorialBF_dowhile(int n) {
5         int fakto = 1;
6         int i = 1;
7         do {
8             fakto = fakto * i;
9             i++;
10        } while (i <= n);
11        return fakto;
12    }
13}

PROBLEMS 12 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Run: MainPangkat06
Masukkan jumlah elemen: 3
Masukkan nilai basis elemen ke-1: 2
Masukkan nilai pangkat elemen ke-1: 3
Masukkan nilai basis elemen ke-2: 4
Masukkan nilai pangkat elemen ke-2: 5
Masukkan nilai basis elemen ke-3: 6
Masukkan nilai pangkat elemen ke-3: 7
HASIL PANGKAT BRUTEFORCE:
2^3: 8
4^5: 1024
6^7: 279936
HASIL PANGKAT DIVIDE AND CONQUER:
2^3: 8
4^5: 1024
6^7: 279936
Activate Windows
Go to Settings to activate Windows.
```

Keduanya menghasilkan output yang sama karena prinsip Brute Force tetap terpenuhi: menghitung satu per satu secara berurutan.

3. Jelaskan perbedaan antara `fakto *= i;` dan `int fakto = n * faktorialDC(n-1);` !

Jawab:

- `fakto *= i` adalah cara iteratif milik Brute Force. Variabel `fakto` dikalikan dengan `i` secara berurutan dari 1 sampai `n` di dalam loop tanpa memanggil fungsi lain. Prosesnya sederhana dan tidak membutuhkan memori tambahan untuk call stack.
- `int fakto = n * faktorialDC(n-1)` adalah cara rekursif milik Divide and Conquer. Fungsi memanggil dirinya sendiri dengan argumen yang lebih kecil (`n-1`) sampai mencapai base case, kemudian hasil setiap pemanggilan dikombinasikan dengan perkalian `n` saat return berjalan balik. Cara ini membentuk tumpukan pemanggilan (call stack) sehingga membutuhkan memori lebih besar dibanding cara iteratif.

4. Buat Kesimpulan tentang perbedaan cara kerja method `faktorialBF()` dan `faktorialDC()`!

Jawab:

- `faktorialBF()` menyelesaikan faktorial dengan cara paling langsung yaitu menggunakan loop yang mengalikan angka satu per satu dari 1 hingga `n`. Cara ini mudah dipahami dan tidak membutuhkan memori tambahan, namun pendekatannya kurang efisien secara algoritma karena berjalan linear.
- `faktorialDC()` menyelesaikan faktorial dengan cara memecah masalah menjadi submasalah yang lebih kecil secara rekursif hingga mencapai base case, lalu menggabungkan hasilnya saat return berjalan balik. Cara ini lebih elegan secara penulisan kode dan mencerminkan pola pikir Divide and Conquer, meskipun membutuhkan memori lebih karena adanya call stack.

5.3 Percobaan 2: Menghitung Hasil Pangkat dengan Algoritma Brute Force dan Divide and Conquer

Kode program:

```
package BruteForceDivideConquer.minggu5;

public class Pangkat06 {
    int nilai, pangkat;

    Pangkat06(int n, int p) {
        nilai = n;
        pangkat = p;
    }

    int pangkatBF(int a, int n) {
        int hasil = 1;
        for (int i = 0; i < n; i++) {
            hasil = hasil * a;
        }
        return hasil;
    }

    int pangkatDC (int a, int n) {
        if (n == 1) {
            return a;
        } else {
            if (n % 2 == 1) {
                return (pangkatDC(a, n/2) * pangkatDC(a, n/2) * a);
            } else {
                return (pangkatDC(a, n/2) * pangkatDC(a, n/2));
            }
        }
    }
}

package BruteForceDivideConquer.minggu5;
import java.util.Scanner;

public class MainPangkat06 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan jumlah elemen: ");
        int elemen = input.nextInt();

        Pangkat06[] png = new Pangkat06[elemen];
        for (int i = 0; i < elemen; i++) {
```

```

        System.out.print("Masukkan nilai basis elemen ke-" + (i +
1) + ": ");
        int basis = input.nextInt();
        System.out.print("Masukkan nilai pangkat elemen ke-" + (i +
1) + ": ");
        int pangkat = input.nextInt();
        png[i] = new Pangkat06(basis, pangkat);
    }

    System.out.println("HASIL PANGKAT BRUTEFORCE:");
    for (Pangkat06 p : png) {
        System.out.println(p.nilai + "^" + p.pangkat + ": " +
p.pangkatBF(p.nilai, p.pangkat));
    }
    System.out.println("HASIL PANGKAT DIVIDE AND CONQUER:");
    for (Pangkat06 p : png) {
        System.out.println(p.nilai + "^" + p.pangkat + ": " +
p.pangkatDC(p.nilai, p.pangkat));
    }
}
}

```

Hasil running:

```

Masukkan jumlah elemen: 3
Masukkan nilai basis elemen ke-1: 2
Masukkan nilai pangkat elemen ke-1: 3
Masukkan nilai basis elemen ke-2: 4
Masukkan nilai pangkat elemen ke-2: 5
Masukkan nilai basis elemen ke-3: 6
Masukkan nilai pangkat elemen ke-3: 7
HASIL PANGKAT BRUTEFORCE:
2^3: 8
4^5: 1024
6^7: 279936
HASIL PANGKAT DIVIDE AND CONQUER:
2^3: 8
4^5: 1024
6^7: 279936
PS C:\Users\Asus\Documents\Algoritma dan Struktur Data\PraktikumASD>

```

Pertanyaan

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!

Jawab:

- pangkatBF() menghitung pangkat dengan cara mengalikan a sebanyak n kali secara berurutan menggunakan loop. Prosesnya berjalan linear sehingga semakin besar nilai pangkat, semakin banyak operasi yang dibutuhkan. Kompleksitasnya adalah $O(n)$.
- pangkatDC() menghitung pangkat dengan cara membagi eksponen menjadi setengahnya setiap rekursi. Masalah a^n dipecah menjadi $a^{(n/2)} * a^{(n/2)}$, lalu masing-masing bagian diselesaikan secara rekursif. Jika pangkat ganjil, ditambah

satu perkalian a lagi. Cara ini jauh lebih efisien karena kompleksitasnya hanya $O(\log n)$.

2. Apakah tahap combine sudah termasuk dalam kode tersebut? Tunjukkan!

Jawab: Sudah ada. Tahap combine adalah proses menggabungkan hasil dari dua submasalah yang sudah diselesaikan, dan itu terlihat pada baris berikut:

→ Combine untuk pangkat genap:

```
return (pangkatDC(a, n/2) * pangkatDC(a, n/2));
```

→ Combine untuk pangkat ganjil:

```
return (pangkatDC(a, n/2) * pangkatDC(a, n/2) * a);
```

Dua hasil dari submasalah `pangkatDC(a, n/2)` digabungkan melalui operasi perkalian untuk membentuk solusi akhir.

3. Pada method `pangkatBF()` terdapat parameter untuk melewati nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class `Pangkat` telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method `pangkatBF()` yang tanpa parameter?

Jawab:

- Method `pangkatBF(int a, int n)` tetap relevan memiliki parameter karena bisa digunakan secara umum untuk menghitung pangkat nilai apapun, tidak terbatas pada atribut objek saja. Namun bisa juga dibuat tanpa parameter karena class `Pangkat` sudah memiliki atribut nilai dan pangkat. Berikut versi tanpa parameter:

```
int pangkatBF() {  
    int hasil = 1;  
    for (int i = 0; i < pangkat; i++) {  
        hasil = hasil * nilai;  
    }  
    return hasil;  
}
```

Versi tanpa parameter lebih sederhana dan langsung memanfaatkan atribut objek, namun kehilangan fleksibilitas untuk menghitung pangkat nilai di luar atribut objek.

4. Tarik tentang cara kerja method `pangkatBF()` dan `pangkatDC()`!

Jawab:

- `pangkatBF()` bekerja secara linear dengan mengalikan basis sebanyak eksponen kali satu per satu. Mudah dipahami namun semakin lambat seiring meningkatnya nilai eksponen karena kompleksitasnya $O(n)$.
- `pangkatDC()` bekerja dengan membagi eksponen menjadi setengahnya setiap langkah rekursi sehingga jumlah operasi jauh berkurang. Contohnya 2^8 cukup

butuh 3 langkah ($8 \rightarrow 4 \rightarrow 2 \rightarrow 1$) dibanding pangkatBF() yang butuh 8 langkah. Kompleksitasnya $O(\log n)$ membuatnya jauh lebih efisien untuk eksponen yang besar.

5.4 Percobaan 3: Menghitung Sum Array dengan Algoritma Brute Force dan Divide and Conquer

Kode program:

```
package BruteForceDivideConquer.minggu5;

public class Sum06 {
    double keuntungan[];

    Sum06(int el) {
        keuntungan = new double[el];
    }

    double totalBF() {
        double total = 0;
        for (int i = 0; i < keuntungan.length; i++) {
            total = total + keuntungan[i];
        }
        return total;
    }

    double totalDC (double arr[], int l, int r) {
        if (l == r) {
            return arr[l];
        }
        int mid = (l + r) / 2;
        double lsum = totalDC(arr, l, mid);
        double rsum = totalDC(arr, mid + 1, r);
        return lsum + rsum;
    }
}

package BruteForceDivideConquer.minggu5;
import java.util.Scanner;

public class MainSum06 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan jumlah elemen: ");
        int elemen = input.nextInt();
    }
}
```

```

Sum06 sm = new Sum06(elemen);
for (int i = 0; i < elemen; i++) {
    System.out.print("Masukkan keuntungan ke-" + (i + 1) + ":
");
    sm.keuntungan[i] = input.nextDouble();
}

System.out.println("Total keuntungan menggunakan Bruteforce: "
+ sm.totalBF());
System.out.println("Total keuntungan menggunakan Divide and
Conquer: " + sm.totalDC(sm.keuntungan, 0, elemen-1));
}
}

```

Hasil running:

```

Masukkan jumlah elemen: 5
Masukkan keuntungan ke-1: 10
Masukkan keuntungan ke-2: 20
Masukkan keuntungan ke-3: 30
Masukkan keuntungan ke-4: 40
Masukkan keuntungan ke-5: 50
Total keuntungan menggunakan Bruteforce: 150.0
Total keuntungan menggunakan Divide and Conquer: 150.0
PS C:\Users\Asus\Documents\Algoritma dan Struktur Data\PraktikumASD>

```

Pertanyaan

1. Kenapa dibutuhkan variable mid pada method TotalDC()?

Jawab: Variabel $mid = (l+r)/2$ dibutuhkan sebagai titik pembagi array menjadi dua bagian yang berukuran hampir sama. Tanpa mid, proses Divide and Conquer tidak bisa berjalan karena array tidak bisa dibagi. Jika pembagian tidak seimbang, performa algoritma akan menurun mendekati worst case.

2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

```

double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);

```

Jawab:

- $lsum = totalDC(arr, l, mid)$ digunakan untuk menghitung total penjumlahan elemen pada bagian kiri array, yaitu dari indeks l sampai mid secara rekursif.
- $rsum = totalDC(arr, mid+1, r)$ digunakan untuk menghitung total penjumlahan elemen pada **bagian kanan** array, yaitu dari indeks mid+1 sampai r secara rekursif.

Keduanya terus memanggil dirinya sendiri dengan ukuran yang semakin kecil hingga mencapai base case. Ini merupakan tahap Conquer dalam Divide and Conquer.

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

```
return lsum+rsum;
```

Jawab: Penjumlahan lsum + rsum diperlukan karena setelah bagian kiri (lsum) dan kanan (rsum) selesai dihitung secara rekursif, hasilnya perlu digabungkan kembali untuk mendapatkan total keseluruhan array. Ini adalah tahap Combine dalam Divide and Conquer.

4. Apakah base case dari totalDC()?

Jawab: Base case dari totalDC() adalah kondisi $l == r$, artinya array hanya tersisa satu elemen sehingga langsung dikembalikan nilainya tanpa rekursi lagi. Tanpa base case, rekursi akan terus berjalan tanpa henti dan menyebabkan Stack Overflow.

5. Tarik Kesimpulan tentang cara kerja totalDC()

Jawab: totalDC() bekerja melalui tiga tahap yaitu Divide membagi array menjadi dua bagian menggunakan mid, Conquer menjumlahkan masing-masing bagian secara rekursif melalui lsum dan rsum hingga mencapai base case $l == r$, dan Combine menggabungkan keduanya dengan return lsum + rsum. Berbeda dengan totalBF() yang menjumlahkan elemen satu per satu, totalDC() memecah masalah menjadi submasalah yang lebih kecil sehingga penyelesaiannya lebih terstruktur dan efisien.